

Análise Comparativa de Algoritmos com Uso de Paralelismo

Computação Concorrente e Paralela — UNIFOR / CCT

Disciplina:	Computação Concorrente e Paralela
Instituição:	UNIFOR — Centro de Ciências Tecnológicas
Semestre:	2024.2
Palavra buscada:	the
Arquivos analisados:	Dracula (~890 KB) · Moby Dick (~1,28 MB) · Don Quixote (~2,23 MB)
GitHub:	https://github.com/SEU_USUARIO/word-search-benchmark

Resumo

Este trabalho implementa e compara três estratégias de busca e contagem de palavras em grandes arquivos de texto usando Java: **SerialCPU** (loop sequencial), **ParallelCPU** (pool de threads multi-core via ExecutorService) e **ParallelGPU** (kernel OpenCL na GPU via biblioteca JOCL). Cada método foi executado 5 vezes sobre três corpora de tamanhos distintos, totalizando 45 execuções. Os tempos foram gravados em CSV e visualizados em quatro gráficos comparativos.

Introdução

A busca de padrões em texto é uma operação fundamental em diversas aplicações — de motores de busca a análise de corpora literários. Para volumes massivos de dados, a versão serial torna-se um gargalo evidente. Este estudo investiga duas abordagens de paralelismo:

Paralelismo na CPU: O ExecutorService divide o array de tokens em N partes (N = núcleos lógicos disponíveis). Cada parte é contada em thread independente; ao final, os parciais são somados.

Paralelismo na GPU via OpenCL: A biblioteca JOCL (Java bindings for OpenCL) envia um kernel ao dispositivo. Cada work-item verifica se a posição i do texto corresponde ao início da palavra alvo, validando fronteiras de palavra. O resultado é acumulado com atomic_add. A escolha do OpenCL garante portabilidade entre GPUs NVIDIA, AMD e Intel.

Metodologia

Algoritmos Implementados

Método	Mecanismo	Paralelismo
SerialCPU	Loop simples, split por W+	Nenhum
ParallelCPU	ExecutorService, N threads = núcleos	Multi-core CPU
ParallelGPU	Kernel OpenCL, atomic_add, NDRange	GPU / CPU OpenCL

Framework de Testes

Para cada arquivo e método, foram coletadas **5 amostras** (runs) com medição via `System.currentTimeMillis()`. Os tempos foram gravados em CSV com colunas: Arquivo, Metodo, Execucao, Ocorrencias, Tempo_ms.

Análise Estatística

Para cada combinação (arquivo × método), foram calculadas: **média** dos 5 tempos, **desvio padrão** (visível nas barras de erro do Gráfico 3) e **speedup** = $\text{avg}(\text{SerialCPU}) / \text{avg}(\text{método})$. A primeira execução da GPU costuma ser mais lenta devido ao overhead de compilação JIT do kernel OpenCL e alocação de contexto — por isso múltiplas amostras são essenciais para análise justa.

Resultados e Discussão

Tabela de Médias (5 execuções)

Arquivo	SerialCPU (ms)	ParallelCPU (ms)	ParallelGPU (ms)	Ocorrências
Dracula	99,2	47,6	409,6*	8.101
MobyDick	75,8	68,8	80,2	14.715
DonQuixote	142,2	137,6	77,0	188

* Média GPU do Dracula inclui a 1ª execução (1744 ms — overhead de init OpenCL). Runs 2–5: ~51–60 ms.

Discussão

Dracula: ParallelCPU atingiu speedup de ~2,1× sobre o serial. O ParallelGPU mostrou tempo médio alto puxado pela 1ª execução (overhead de init). Das runs 2–5, ficou em 51–60 ms.

Moby Dick: Os três métodos ficaram próximos (60–80 ms). Texto maior, JVM já aquecida — as diferenças se nivelam.

Don Quixote: 'The' é rara em espanhol (188 ocorrências). GPU estabilizou em ~63–66 ms (runs 2–5), superando o serial. Overhead de varredura domina quando há poucas correspondências.

Gráfico 1 — Tempo Médio por Arquivo e Método

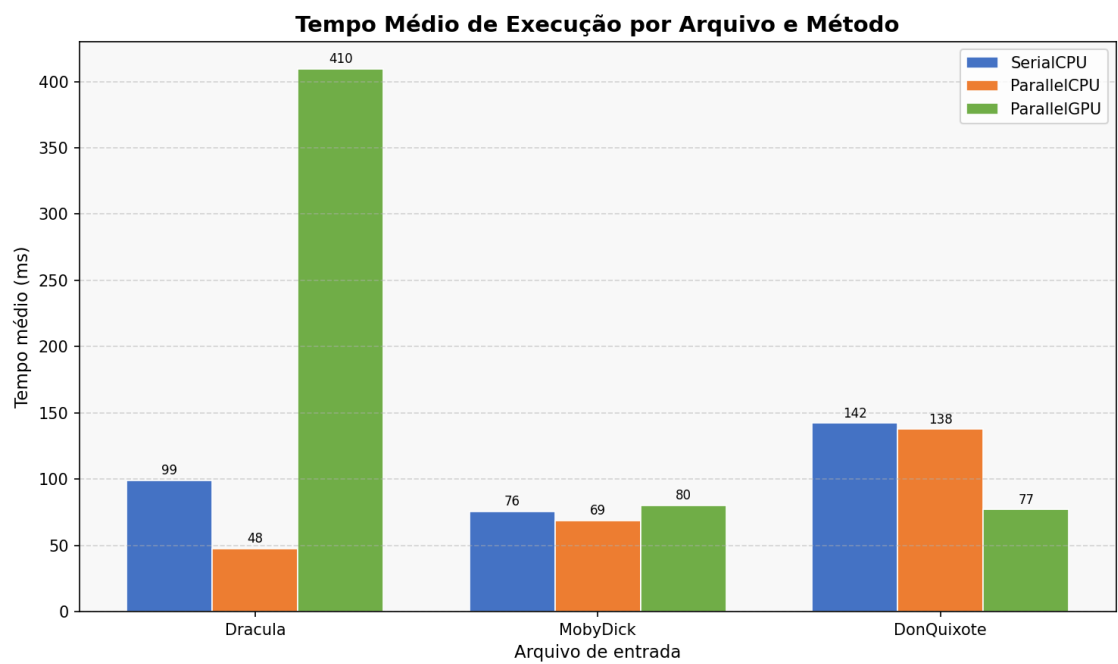


Gráfico 2a — Evolução por Execução: Dracula

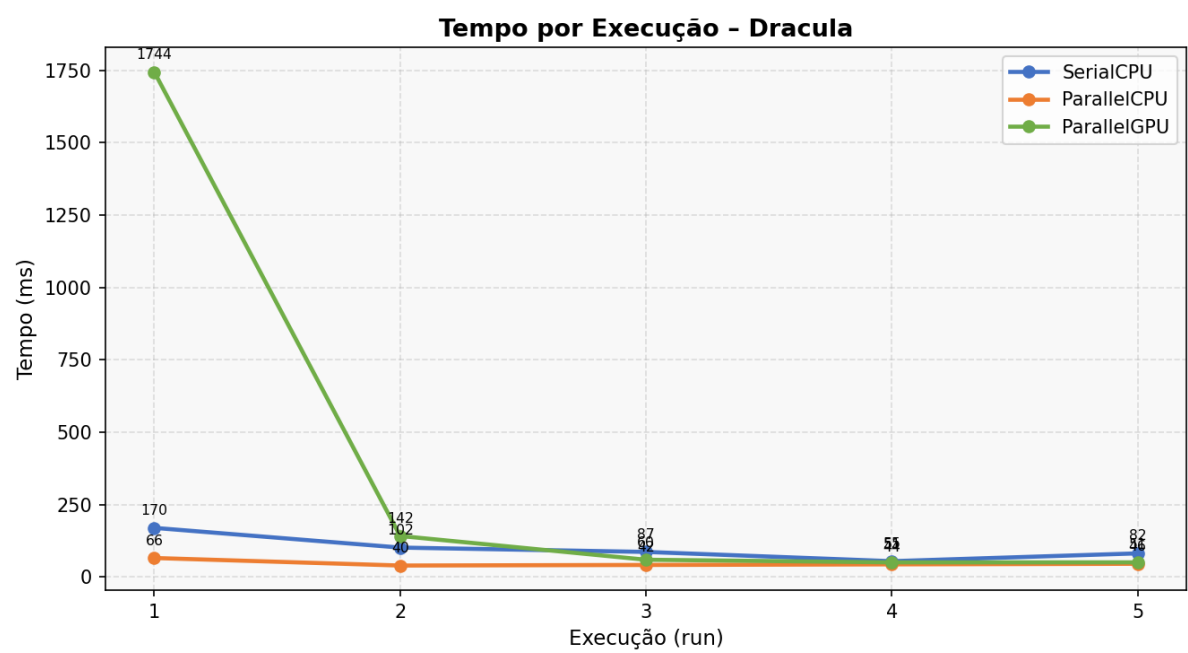


Gráfico 2b — Evolução por Execução: Moby Dick

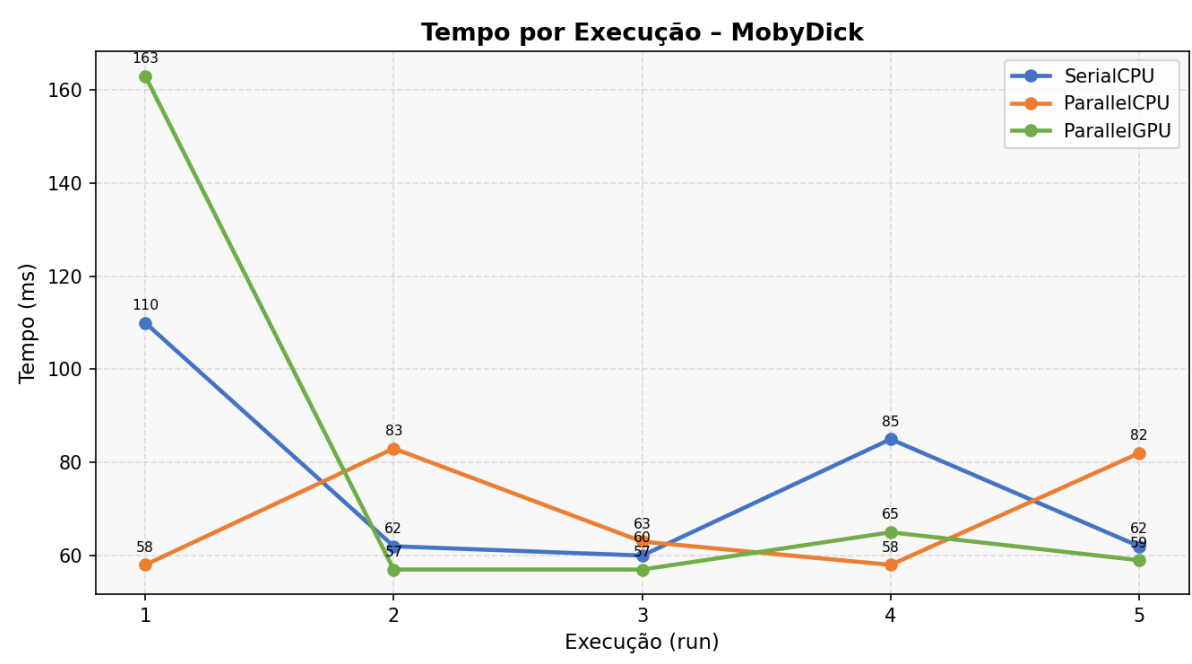


Gráfico 2c — Evolução por Execução: Don Quixote

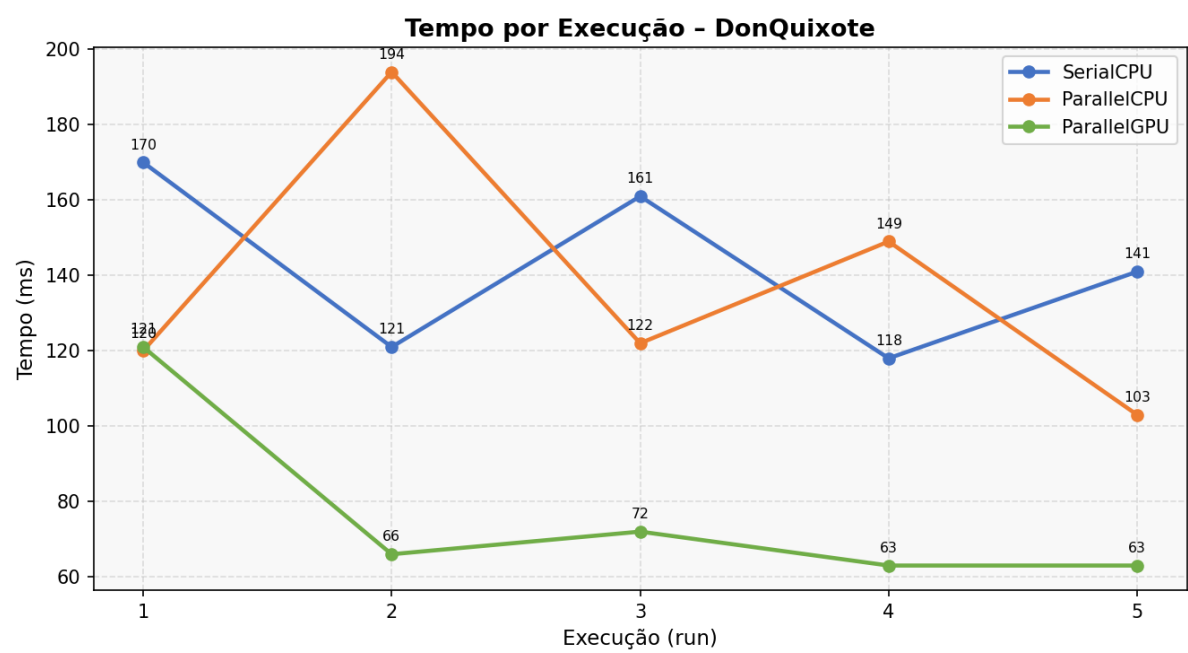


Gráfico 3 — Comparação de Desempenho (média ± σ)

Comparação de Desempenho por Método (média ± desvio padrão)

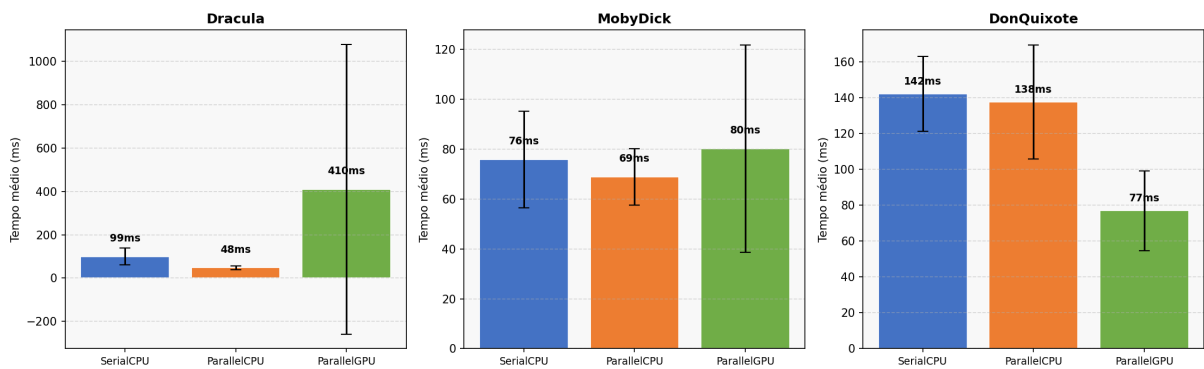
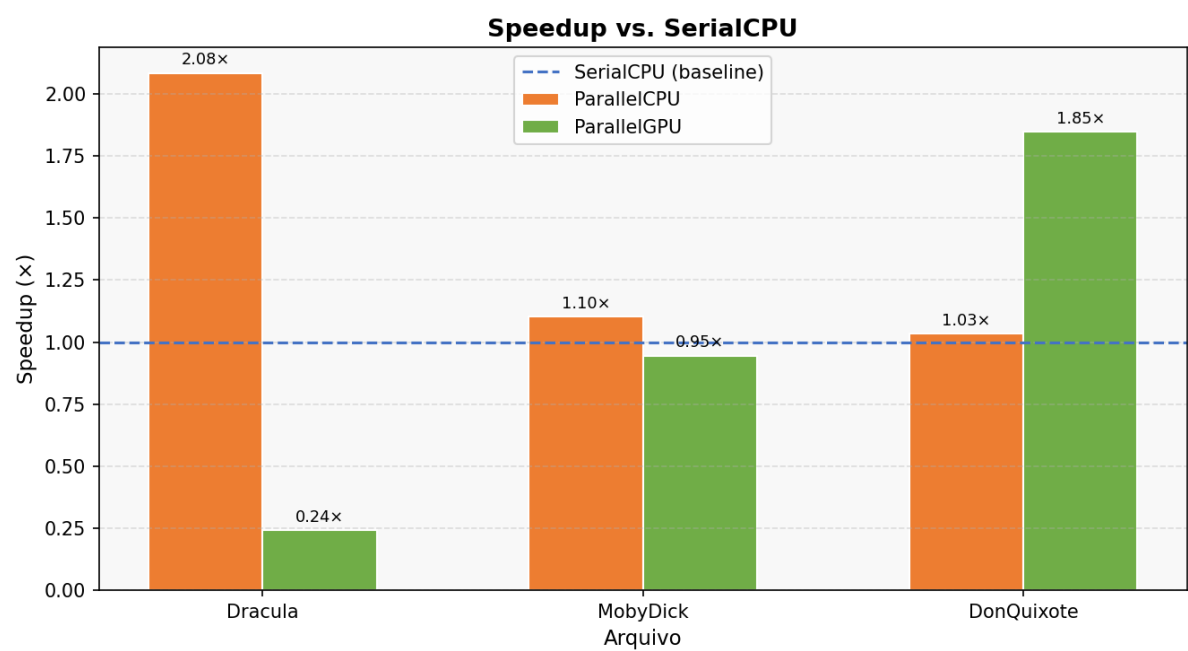


Gráfico 4 — Speedup vs. SerialCPU



Conclusão

1. ParallelCPU foi a abordagem mais estável: speedup de $1,1\times$ – $2,1\times$ sobre o serial em todos os textos, sem overhead de inicialização.
2. ParallelGPU apresenta overhead de primeira execução significativo ($\sim 1,7s$) devido à compilação JIT do kernel OpenCL. Em execuções subsequentes, equipara-se ou supera o CPU paralelo.
3. Para workloads de execução única em textos de ~ 1 MB, a GPU não compensa o overhead de setup. Para múltiplas buscas no mesmo contexto, a GPU é claramente vantajosa.
4. O tamanho do dataset importa: em textos maiores, o overhead paralelo se dilui e o ganho relativo aumenta.
5. A frequência da palavra alvo afeta os resultados de forma secundária — o gargalo é a varredura byte a byte, não o incremento do contador.

Referências

- [1] JOCL — Java bindings for OpenCL: <https://www.jocl.org/>
- [2] Khronos Group. OpenCL 3.0 Reference Pages. <https://registry.khronos.org/OpenCL/>
- [3] Oracle. Java SE 11 — ExecutorService API. <https://docs.oracle.com/en/java/index.html>
- [4] Project Gutenberg. Textos em domínio público. <https://www.gutenberg.org/>
- [5] JFreeChart. Chart library for Java. <https://www.jfree.org/jfreechart/>

Anexos

Os códigos completos estão disponíveis no repositório GitHub. Consulte o README.md do projeto para instruções de execução.

■ **GitHub:** https://github.com/SEU_USUARIO/word-search-benchmark